

Makefile Python pour Windows

Ce projet utilise un **Makefile** pour simplifier la création et l'utilisation d'un environnement virtuel Python sous Windows.

L'objectif est de pouvoir préparer, lancer, nettoyer et diagnostiquer un projet Python avec quelques commandes simples :

```
make setup PY=3.10
make run
make shell
make clean
```

Le Makefile mémorise également la dernière version de Python sélectionnée pour le projet.

1. Prérequis

Python

Python doit être installé avec le **Python Launcher pour Windows** (`py.exe`).

Vérification :

```
py --version
```

Lister les versions installées :

```
py -0p
```

Le Makefile permet ensuite de choisir précisément la version utilisée :

```
make setup PY=3.10
```

GNU Make

La commande `make` doit être installée et accessible dans le `PATH`.

Vérification :

```
make --version
```

PowerShell

PowerShell est utilisé par certaines commandes du Makefile, notamment :

```
make shell
```

et pour certaines opérations de nettoyage.

Windows 10 et Windows 11 possèdent normalement PowerShell par défaut.

2. Arborescence minimale attendue

Le Makefile actuel suppose par défaut cette organisation :

```
mon_projet/  
|  
├─ Makefile  
├─ requirements.txt  
└─ src/  
    └─ main.py
```

Le `Makefile` doit se trouver à la **racine du projet**.

Le point d'entrée par défaut est :

```
src/main.py
```

Le fichier de dépendances par défaut est :

```
requirements.txt
```

3. Prérequis liés à l'arborescence

Pour utiliser le Makefile sans le modifier :

- `Makefile` doit être situé à la racine ;
- `src/main.py` doit exister pour utiliser `make run` ;
- `requirements.txt` doit être à la racine si le projet possède des dépendances ;
- Python doit être accessible avec la commande `py` ;
- GNU Make doit être accessible avec la commande `make`.

Le Makefile crée lui-même les éléments suivants :

```
py3.10/  
.make-python-version
```

si Python 3.10 est sélectionné.

Il n'est donc pas nécessaire de créer manuellement le venv.

4. Structure recommandée pour un petit projet

Pour un petit programme :

```
mon_projet/  
|  
├─ Makefile  
├─ README.md  
├─ requirements.txt  
├─ .gitignore  
└─  
    └─ src/  
        └─ main.py
```

Cette structure convient très bien pour un projet simple.

5. Structure recommandée pour un projet plus important

Dès que le programme commence à grossir, il est préférable de séparer les responsabilités :

```
mon_projet/  
|
```

```

├─ Makefile
├─ README.md
├─ requirements.txt
├─ requirements.lock.txt
├─ pyproject.toml
├─ .gitignore
├─
├─ src/
│   └─ mon_projet/
│       ├── __init__.py
│       ├── main.py
│       ├── config.py
│       ├── utils.py
│       └─ modules/
│           ├── __init__.py
│           ├── module_a.py
│           └─ module_b.py
├─
├─ tests/
│   ├── test_module_a.py
│   └─ test_module_b.py
├─
├─ data/
│   ├── input/
│   └─ output/
├─
└─ docs/

```

Une règle simple à retenir :

`main.py` orchestre le programme ; les autres modules réalisent le travail.

Par exemple :

```

from modules.ocr import extract_text
from modules.translator import translate_text

def main():
    text = extract_text("image.png")
    translated_text = translate_text(text)

    print(translated_text)

if __name__ == "__main__":
    main()

```

Cela évite d'avoir un énorme `main.py` contenant toute la logique du programme.

6. Adapter le Makefile à une autre structure

Le Makefile utilise actuellement :

```
SRC ?= src/main.py
```

Si le point d'entrée devient :

```
src/mon_projet/main.py
```

il est possible de lancer :

```
make run SRC=src/mon_projet/main.py
```

Ou de modifier directement le Makefile :

```
SRC ?= src/mon_projet/main.py
```

Même principe pour `requirements.txt` :

```
REQ ?= requirements.txt
```

Un autre fichier peut être utilisé avec :

```
make install REQ=config/requirements.txt
```

7. Premier lancement

Pour préparer complètement le projet avec Python 3.10 :

```
make setup PY=3.10
```

Cette commande :

1. vérifie que Python 3.10 est installé ;
2. crée l'environnement `py3.10` ;

3. met à jour `pip`, `setuptools` et `wheel` lors de sa création ;
4. installe `requirements.txt` ;
5. mémorise Python 3.10 comme version du projet.

Après cela :

```
make run
```

suffit.

8. Mémorisation de Python

Après :

```
make setup PY=3.10
```

le Makefile crée automatiquement :

```
.make-python-version
```

avec :

```
PY := 3.10
```

Toutes les commandes suivantes utiliseront alors Python 3.10 :

```
make run
make install
make shell
make clean
make rebuild
make info
```

Il n'est donc pas nécessaire d'écrire à chaque fois :

```
PY=3.10
```

Pour changer de version :

```
make setup PY=3.12
```

Python 3.12 devient alors la version mémorisée.

9. Créer uniquement le venv

```
make venv PY=3.10
```

Le Makefile crée :

```
py3.10/
```

L'interpréteur Python correspondant est :

```
py3.10/Scripts/python.exe
```

10. Lancer le programme

```
make run
```

Le Makefile lance par défaut :

```
src/main.py
```

avec l'interpréteur du venv.

Avec Python 3.10, cela revient conceptuellement à :

```
.\py3.10\Scripts\python.exe src\main.py
```

Il n'est donc **pas nécessaire d'activer manuellement le venv** avant `make run`.

11. Vérifier que le venv est utilisé

Pour vérifier quel Python est réellement utilisé :

```
import sys

print(sys.executable)
```

Avec Python 3.10, le résultat doit ressembler à :

```
C:\...\mon_projet\py3.10\Scripts\python.exe
```

12. Passer des arguments au programme

Le Makefile possède la variable `ARGS` .

Exemple :

```
make run ARGS="--debug --port 8080"
```

Cela revient à exécuter :

```
python src/main.py --debug --port 8080
```

avec le Python du venv.

13. Installer les dépendances

```
make install
```

Le Makefile lit :

```
requirements.txt
```

et installe les dépendances dans le venv.

Il utilise l'équivalent de :

```
py3.10/Scripts/python.exe -m pip
```

et non le `pip` global de Windows.

14. requirements.txt

Exemple :

```
# OCR
easyocr==1.7.1

# Traitement d'images
Pillow==10.1.0
numpy==1.24.3
opencv-python==4.8.1.78

# Traduction
googletrans==4.0.0rc1

# Utilitaires
tqdm==4.66.1

# GPU
torch
torchvision
```

Les commentaires doivent commencer par :

```
#
```

Il ne faut pas utiliser :

```
"""
Commentaire
"""
```

car `requirements.txt` n'est pas interprété comme un fichier Python.

15. Réinstaller les dépendances

```
make reinstall
```

Cette commande force la réinstallation des dépendances.

Elle peut être utile lorsqu'un environnement semble corrompu.

16. Mettre à jour les outils Python

```
make upgrade
```

Cette commande met à jour :

- pip
- setuptools
- wheel

dans l'environnement virtuel.

17. Générer un fichier de verrouillage

```
make freeze
```

Le Makefile génère :

```
requirements.lock.txt
```

Ce fichier contient les versions exactes installées dans l'environnement.

Exemple :

```
numpy==1.24.3
Pillow==10.1.0
torch==2.14.0
...
```

18. `make shell`

```
make shell
```

Cette commande ouvre un **nouveau PowerShell** avec le venv activé.

Le terminal ressemble alors à :

```
(py3.10) PS C:\...\mon_projet>
```

Dans ce terminal :

```
python --version
pip list
python src/main.py
pip install requests
```

utilisent directement le venv.

`make shell` est surtout utile lorsqu'on souhaite travailler manuellement dans l'environnement.

Il n'est pas nécessaire pour :

```
make run
```

19. Quitter `make shell`

Comme `make shell` ouvre un nouveau PowerShell, utiliser :

```
exit
```

pour le fermer.

Si le venv a été activé manuellement dans un terminal existant :

```
deactivate
```

20. `make off`

```
make off
```

ne peut pas directement désactiver le venv du terminal parent.

Le Makefile affiche donc simplement les instructions :

```
Après make shell :
    exit
```

```
Après activation manuelle :  
deactivate
```

21. Sélectionner Python sans créer de venv

```
make select-python PY=3.10
```

Cette commande mémorise Python 3.10 dans :

```
.make-python-version
```

sans créer immédiatement `py3.10` .

22. Oublier la version mémorisée

```
make reset-python
```

Cela supprime :

```
.make-python-version
```

Le Makefile revient alors à sa valeur par défaut :

```
Python 3.11
```

23. Voir la configuration

```
make info
```

Cette commande affiche notamment :

- version Python sélectionnée ;
- état de la mémoire du projet ;
- dossier du venv ;
- chemin de Python ;
- script principal ;

- fichier requirements ;
 - arguments ;
 - présence ou absence du venv.
-

24. Voir les Python installés

```
make list-python
```

Cette commande est utile avant :

```
make setup PY=3.10
```

pour vérifier que la version demandée existe.

25. Supprimer le venv

```
make clean
```

Si Python 3.10 est sélectionné, le Makefile supprime uniquement :

```
py3.10/
```

Il ne supprime pas tous les dossiers `py3*`.

26. `make clear`

```
make clear
```

est simplement un alias de :

```
make clean
```

27. Reconstruire le venv

```
make rebuild
```

Cette commande réalise :

```
suppression du venv
↓
création du venv
↓
installation des dépendances
```

Elle est pratique lorsqu'on souhaite repartir sur un environnement propre.

28. Afficher l'aide

```
make help
```

ou simplement :

```
make
```

`help` est la cible par défaut du Makefile.

29. Workflow recommandé

Nouveau clone

```
git clone <url-du-projet>
cd mon_projet

make setup PY=3.10
make run
```

Utilisation quotidienne

```
make run
```

Travailler manuellement

```
make shell
```

Puis :

```
python --version  
pip list
```

Pour quitter :

```
exit
```

Ajouter une dépendance

Ajouter le package dans :

```
requirements.txt
```

puis :

```
make install
```

30. Fichiers générés automatiquement

Le projet peut contenir localement :

```
py3.10/  
.make-python-version  
requirements.lock.txt  
__pycache__/  
.pytest_cache/
```

31. `.gitignore` recommandé

```
# Environnements virtuels  
py3*/
```

```
# Cache Python
__pycache__/
*.py[cod]
*$py.class

# Tests
.pytest_cache/
.coverage
htmlcov/

# IDE
.vscode/
.idea/

# Variables d'environnement / secrets
.env
.env.*

# Fichiers temporaires
*.log
*.tmp

# Build Python
build/
dist/
*.egg-info/
```

Pour `.make-python-version`, deux possibilités existent.

Configuration locale

Ajouter :

```
.make-python-version
```

Chaque développeur choisit alors sa propre version.

Configuration commune

Ne pas ignorer :

```
.make-python-version
```

La version sélectionnée peut alors être partagée dans Git.

32. pyproject.toml

Pour un projet Python moderne, on peut également utiliser :

```
pyproject.toml
```

Exemple :

```
[project]
name = "mon-projet"
version = "0.1.0"
requires-python = ">=3.10,<3.13"

dependencies = []
```

Les rôles sont différents :

- **Makefile** : automatise les commandes ;
- **pyproject.toml** : décrit et configure le projet Python ;
- **requirements.txt** : liste les dépendances ;
- **README.md** : explique le fonctionnement du projet.

33. Arborescence complète recommandée

```
mon_projet/
├── Makefile
├── README.md
├── requirements.txt
├── pyproject.toml
├── .gitignore
├── src/
│   ├── mon_projet/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── config.py
│   │   └── utils.py
│   └── modules/
│       ├── __init__.py
│       ├── module_a.py
│       └── module_b.py
└── tests/
```

```
|   |─ test_module_a.py
|   |─ test_module_b.py
|   |
|   └─ data/
|       ├── input/
|       └─ output/
|   |
|   └─ docs/
```

Après :

```
make setup PY=3.10
```

on aura également :

```
mon_projet/
|
|─ py3.10/
|─ .make-python-version
|
└─ ...
```

34. Adapter `make run` à cette structure

Si le programme principal est :

```
src/mon_projet/main.py
```

modifier :

```
SRC ?= src/main.py
```

en :

```
SRC ?= src/mon_projet/main.py
```

Ou ponctuellement :

```
make run SRC=src/mon_projet/main.py
```

35. Récapitulatif des commandes

Commande	Fonction
<code>make</code>	Affiche l'aide
<code>make help</code>	Affiche l'aide
<code>make setup PY=3.10</code>	Prépare complètement le projet
<code>make venv PY=3.10</code>	Crée le venv
<code>make run</code>	Lance le programme
<code>make shell</code>	Ouvre un PowerShell dans le venv
<code>make install</code>	Installe les dépendances
<code>make reinstall</code>	Réinstalle les dépendances
<code>make upgrade</code>	Met à jour pip/setuptools/wheel
<code>make freeze</code>	Génère <code>requirements.lock.txt</code>
<code>make info</code>	Affiche la configuration
<code>make list-python</code>	Liste les versions Python
<code>make select-python PY=3.10</code>	Mémoire une version
<code>make reset-python</code>	Efface la version mémorisée
<code>make clean</code>	Supprime le venv sélectionné
<code>make clear</code>	Alias de <code>make clean</code>
<code>make rebuild</code>	Reconstruit le venv
<code>make off</code>	Explique comment sortir du venv

36. Résumé rapide

Premier lancement :

```
make setup PY=3.10
```

Utilisation quotidienne :

```
make run
```

Ouvrir un terminal dans le venv :

```
make shell
```

Afficher la configuration :

```
make info
```

Repartir avec un venv propre :

```
make rebuild
```

Le Makefile permet ainsi de conserver un environnement Python cohérent et reproductible tout en évitant d'avoir à gérer manuellement le venv à chaque utilisation.